

Predictive KV Cache Amortization for Edge Language Model Inference on Mobile Devices

Arjit Tripathi

arjit.tripathi2023@vitstudent.ac.in

Medha Sriram

medha.sriram2023@vitstudent.ac.in

Heer Shah

heer.shah2023@vitstudent.ac.in

Abstract—Context-aware language model inference on mobile devices requires processing environmental prompts before generating responses, and the cost of this prefill phase scales linearly with prompt length. We present Context-Aware Predictive KV-Cache Priming (CAP-KVC), a system that amortizes prefill costs by using environmental sensor signals—GPS, Bluetooth Low Energy proximity, and time of day—to predict the user’s likely context and pre-compute the corresponding key-value cache tensors during idle periods. At inference time, the pre-computed cache is restored from persistent storage in 2.74 ms on average, bypassing the context prefill entirely. Evaluated on a OnePlus 11R with Qwen2.5-0.5B-Instruct (Q4_K_M), CAP-KVC reduces end-to-end inference latency by $4.24\times$ relative to inline context processing at approximately 500 tokens. Native-side prefill decomposition via `std::chrono` instrumentation isolates the prefill phase from the generation phase, revealing a $40.78\times$ prefill-specific speedup: CAP-KVC processes only 9 intent tokens (343 ms prefill) versus 439 tokens for RAG_INLINE (14001 ms prefill) at the 500-token context tier. The system is implemented as a complete Android application with a C++/JNI inference backend based on `llama.cpp`, a bounded KV cache residency manager supporting three simultaneous cache states within a 118.7 MB native heap budget, and sensor-driven context routing with hysteresis-gated drift detection. A proof-of-concept multimodal intent layer combining surface electromyography and gaze tracking through late fusion is included as a motivating application interface; its evaluation uses synthetic training data and does not constitute clinical validation. All measurements, statistical tests, and limitations—including single-device evaluation—are reported.

Index Terms—Key-value cache, edge inference, predictive caching, large language model, mobile systems, sensor fusion, Android, augmentative and alternative communication.

I. INTRODUCTION

Post-training quantization and optimized inference runtimes have made it feasible to run sub-billion-parameter language models on mobile system-on-chip (SoC) hardware [4], [5], [2]. A key application of on-device language models is context-aware response generation, where an environmental system prompt—encoding the user’s location, schedule, nearby devices, or medical profile—is prepended to the user’s input before inference. This contextual framing improves response relevance but introduces a prefill cost: for a system prompt of N tokens, the model must process the full prompt through the transformer [29] before generating any output. The prefill phase scales as $O(N)$ and, for prompts of several hundred

tokens, can dominate the user-perceived latency on resource-constrained hardware.

Assistive communication provides a concrete motivating scenario. Augmentative and alternative communication (AAC) devices for individuals with conditions such as amyotrophic lateral sclerosis (ALS) require sub-second responsiveness to support conversational turn-taking [1]. A context-aware language model can generate environment-specific responses—medical terminology in a hospital ward, casual greetings when a caregiver arrives—but the prefill cost of the contextual prompt directly competes with the latency budget.

In server-side deployments, KV cache reuse techniques such as prefix sharing [10], paged attention [11], and prompt caching [13] amortize prefill costs across concurrent requests from multiple users. These techniques exploit the shared request stream of multi-tenant serving and are not directly applicable to single-user edge devices, where each inference request typically reprocesses the entire contextual prompt.

This paper presents **Context-Aware Predictive KV-Cache Priming (CAP-KVC)**, a system that decouples the context prefill from the inference request path on mobile hardware. CAP-KVC uses environmental sensor signals—GPS location, Bluetooth Low Energy (BLE) proximity beacons, and time of day—to predict which semantic context the user is most likely to need. During idle periods, the system executes the prefill for the predicted context, serializes the resulting key-value (KV) cache tensors to persistent storage, and registers the context as resident. At inference time, the pre-computed cache is restored from storage and the model processes only the user’s short input tokens, reducing the inference-time cost from a prompt-length-dependent prefill to a cache restoration operation that completes in under 3 ms.

The system is implemented as a complete Android application with a C++/JNI inference backend based on `llama.cpp` [2], a Kotlin orchestration layer managing cache residency and sensor-driven routing, and a proof-of-concept multimodal intent classification layer combining surface EMG and gaze tracking inputs through late fusion.

The contributions of this paper are as follows:

- 1) **Predictive KV Cache Priming Architecture:** A system that amortizes context prefill costs on mobile hardware by using environmental sensor signals for background cache state selection and persistence. On a OnePlus 11R with Qwen2.5-0.5B-Instruct (Q4_K_M), native-side pre-

fill decomposition confirms a $40.78\times$ prefill-specific speedup at $N \approx 500$ tokens (343 ms vs. 14001 ms), and a $4.24\times$ end-to-end speedup (4574 ms vs. 19408 ms). Cache restoration completes in 2.74 ms on average.

- 2) **Edge-Native Android Implementation:** A deployable implementation comprising a scalar-only JNI boundary (8 application-facing native functions, plus a lifecycle cleanup function), a reentrant-lock concurrency model for safe native state access, an LRU cache residency manager bounded to 3 simultaneous KV states within a 118.7 MB native heap budget, and a deterministic context routing engine using a convex sensor scoring function.
- 3) **Sensor-Driven Context Routing:** A closed-form scoring function that dynamically combines temporal, spatial, and BLE proximity signals with reliability-adaptive weighting (time baseline weight $w_t = 0.4$; GPS and BLE weights computed from hardware signal quality), including hysteresis-gated drift detection to prevent cache thrashing under noisy sensor conditions.
- 4) **Proof-of-Concept Multimodal Intent Layer:** A late fusion architecture combining CNN-LSTM sEMG embeddings with MediaPipe gaze tracking, deployed on-device via ONNX Runtime. This component validates architectural compatibility between heterogeneous embedding spaces ($\mathbb{R}^{64}$ and $\mathbb{R}^5$) on synthetic evaluation data and is presented as a proof-of-concept for future clinical validation, not as a validated intent-recognition system.

A. Research Questions

This work addresses the following research questions:

- **RQ1** (Prefill Amortization): To what extent does predictive KV cache priming reduce end-to-end inference latency relative to inline context processing on mobile hardware, and what is the runtime cost of cache restoration from persistent storage?
- **RQ2** (Memory Overhead): What native heap overhead is required to maintain bounded multi-state KV cache residency on a representative mobile device, and is this budget feasible alongside normal application operation?
- **RQ3** (Fusion Architecture Selection): Can a late fusion architecture successfully combine heterogeneous sensor embeddings for intent classification under controlled synthetic evaluation, and does it meet a pre-specified decision criterion relative to cross-attention?

The remainder of this paper is organized as follows. Section II surveys related work in KV cache optimization, edge LLM deployment, and assistive technology. Section III describes the CAP-KVC methodology. Section IV details the Android implementation. Section V presents the experimental setup. Section VI reports and discusses the results. Section VII concludes with a summary of findings and limitations.

II. RELATED WORK

A. KV Cache Optimization

The key-value (KV) cache stores the attention key and value tensors computed during the prefill phase, allowing subsequent tokens to attend to the full context without recomputation. Several lines of research have sought to reduce the memory and computational costs of KV cache management.

Cache compression. Scissorhands [7] identifies and removes KV pairs corresponding to tokens with low cumulative attention scores. H₂O [8] retains high-attention “heavy hitter” tokens while evicting the rest. SnapKV [20] exploits per-head attention consistency to select cluster-level important positions. StreamingLLM [9] maintains a fixed-size attention window plus initial “attention sink” tokens for unbounded sequence lengths. KVQuant [24] applies non-uniform quantization to KV cache tensors, enabling million-token context lengths on a single GPU. These methods operate at the token level within a single sequence and are complementary to the state-level caching proposed here.

Prompt caching and prefix reuse. SGLang’s RadixAttention [10] organizes cached prefixes in a radix tree for server-side multi-tenant reuse. vLLM’s PagedAttention [11] shares KV blocks across concurrent requests with common prefixes. Prompt Cache [21] precomputes and stores attention states for reusable prompt modules, achieving up to $8\times$ TTFT reduction on GPU. CacheBlend [22] extends prefix reuse to non-prefix-aligned chunks by selectively recomputing a small fraction of affected token positions. These systems target cloud inference serving with shared request streams and do not address single-user edge devices.

Cross-session state management. InfiniGen [12] speculatively prefetches KV cache entries for predicted future tokens. CacheGen [13] compresses and streams KV caches for reuse across sessions. The present work is most closely aligned with this family, differing in its use of environmental sensor signals—rather than textual context or speculative token prediction—for cache state selection on a single edge device.

B. Edge Deployment of Language Models

Post-training quantization to 4-bit precision [4], [5] reduces both memory footprint and arithmetic intensity, enabling sub-billion-parameter models to run on smartphone SoCs. AWQ [25] further improves 4-bit weight-only quantization by protecting salient weights based on activation-aware scaling. MobileLLM [26] demonstrates that sub-billion-parameter architectures with depth-prioritized design can achieve competitive quality for on-device tasks. Inference frameworks such as llama.cpp [2] and MLC-LLM [6] provide ARM NEON-optimized kernels and memory-mapped model loading for mobile execution. PowerInfer-2 [23] extends on-device inference to models up to 47B parameters through neuron-cluster-level heterogeneous scheduling across NPU and CPU, though it targets a different scale regime than the sub-billion-parameter models considered here. Alizadeh et al. [27] address the memory bandwidth bottleneck for on-device inference by using flash memory and windowed KV cache access, demonstrating that models exceeding DRAM capacity can still

achieve interactive latency. FlexGen [28] explores offloading-based inference across GPU, CPU, and disk for throughput-oriented settings.

Despite runtime improvements, the prefill phase of autoregressive inference remains a latency bottleneck on mobile hardware. For a system prompt of N tokens, the prefill requires processing all N tokens through the transformer before the first output token can be generated. When contextual prompts span several hundred tokens—as in environment-aware applications—this cost can dominate user-perceived latency.

C. Augmentative and Alternative Communication

AAC devices serve individuals with severe motor impairments, including those caused by amyotrophic lateral sclerosis (ALS), cerebral palsy, and acquired brain injuries [1]. Recent work has explored integrating language models into AAC workflows for word and phrase prediction [16], context-sensitive sentence completion, and open-vocabulary generation. Deploying language models on edge devices introduces a tension between response quality and latency that motivates the prefill amortization approach presented here.

D. Multimodal Sensor Fusion

Combining EMG and gaze signals for human-computer interaction has been explored in assistive technology contexts. EMG-based gesture classification has been evaluated on public datasets including NinaPro [14]. Gaze-based interaction using facial landmark models such as MediaPipe [15] provides a complementary input channel. Late fusion and cross-attention architectures have been compared for combining heterogeneous sensor modalities [17], with late fusion often preferred when modality reliability varies.

E. Positioning of This Work

The present work differs from prior KV cache optimization research in two respects. First, it targets a single-user, single-device edge deployment where background computation during idle periods can be exploited for prefill amortization—an operational model unavailable in cloud serving. Prompt Cache [21] and CacheBlend [22] assume a server-side setting where multiple requests share cached state; CAP-KVC instead serializes KV state to persistent storage for restoration by the same user across sessions. Second, it introduces environmental sensor signals (GPS, BLE proximity, time of day) as the basis for predictive cache state selection, replacing the textual prefix-matching strategies used in server-side systems.

III. METHODOLOGY

This section describes the design of the Context-Aware Predictive KV-Cache Priming (CAP-KVC) system and its integration within the AACBridge communication pipeline. The primary goal is to amortize the computational cost of contextual prompt ingestion on resource-constrained mobile hardware by executing it as a background process, prior to user interaction. Fig. 1 presents an overview of the system architecture.

A. Problem Formulation

Consider a user u operating an AAC device in a physical environment characterized by temporal, spatial, and proximity signals. Let $C = \{c_1, c_2, \dots, c_M\}$ denote a finite set of pre-configured semantic contexts, each associated with a textual system prompt p_i of length N_i tokens. In conventional retrieval-augmented generation (RAG) pipelines, the system prompt p_i is concatenated with the user’s intent tokens and submitted to the language model at inference time, incurring an $O(N_i)$ prefill cost that directly adds to user-perceived latency.

CAP-KVC reformulates this as a scheduling problem: given a prediction of the user’s most likely context $\hat{c}$ based on environmental signals, execute the prefill of $p_{\hat{c}}$ during idle periods and persist the resulting key-value (KV) cache tensors to persistent storage. At inference time, the system restores the pre-computed KV cache and appends only the user’s intent tokens, reducing the inference-time cost from a prompt-length-dependent prefill to a cache restoration operation whose cost is independent of the original prompt length N_i .

B. System Execution Lifecycle

The system operates through a five-stage pipeline, where the first three stages execute asynchronously during background idle periods and the final two stages execute at user-interaction time:

- 1) **Sensor Acquisition.** The system concurrently polls available hardware sensors—GPS, Bluetooth Low Energy (BLE), and the system clock—to construct an immutable `SensorSnapshot` representing the user’s current environmental state.
- 2) **Deterministic Context Routing.** The snapshot is evaluated by a `StateRouter` against all registered semantic contexts using a convex scoring function (Section III-D). The top- k highest-scoring contexts are selected for cache residency, where k is bounded by a device-specific RAM constraint (Section III-E).
- 3) **Background Cache Priming.** For each selected context not already resident in memory, the system executes a background prefill of its system prompt through the language model, serializes the resulting KV cache tensors to persistent storage, and registers the context as resident.
- 4) **Resident Inference.** Upon user interaction, the pre-computed KV cache for the predicted context is restored from storage into native memory. The user’s intent tokens are then appended and decoded, bypassing the context prefill entirely.
- 5) **Drift Correction.** An asynchronous background daemon periodically re-evaluates environmental scores to detect changes in the user’s physical context and triggers cache replacement when a new context scores sufficiently higher than the weakest resident context (Section III-F).

C. Multimodal Intent Classification

User intent is derived from two complementary input modalities, fused to produce a single classification decision that is then formatted as a textual prompt for the language model.

CAP-KVC System Architecture

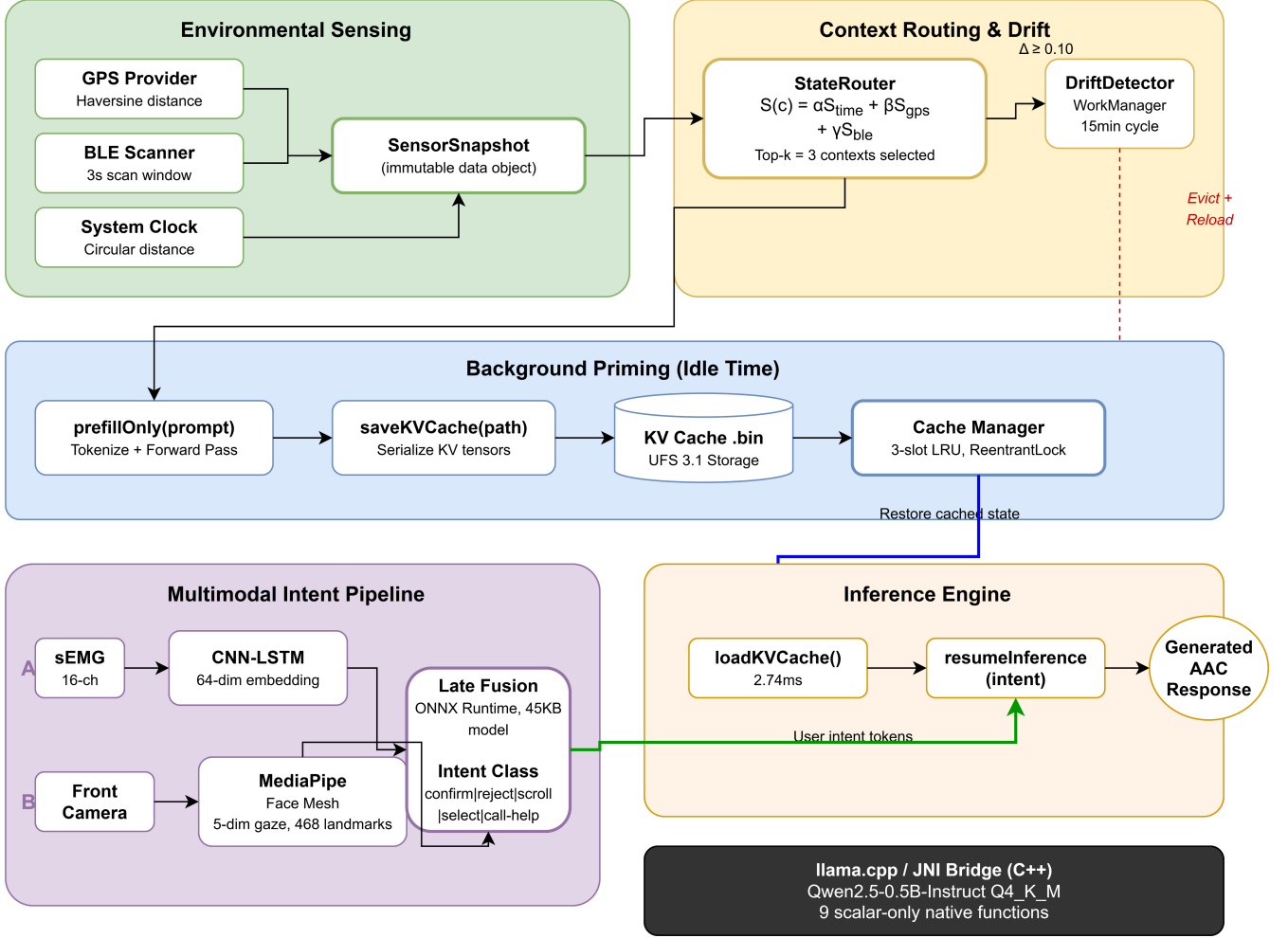


Fig. 1. CAP-KVC system architecture. Environmental sensors (blue) feed a deterministic scoring function that selects contexts for background priming (green). The multimodal intent pipeline (purple) combines sEMG and gaze inputs via late fusion. At inference time (orange), the pre-computed KV cache is restored and only intent tokens are processed by the llama.cpp/JNI backend. A periodic drift detector re-evaluates context relevance with hysteresis gating ($\Delta \geq 0.10$).

1) *Surface Electromyography (sEMG)*: A facial sEMG wearable provides 16-channel signals sampled continuously. Raw signals are segmented into fixed-length windows and processed by a CNN-LSTM classifier that extracts a 64-dimensional embedding vector. The CNN block applies three sequential Conv1d–BatchNorm–ReLU–MaxPool layers for temporal feature extraction, followed by a two-layer LSTM for temporal context modeling. The final hidden state is projected to a 64-dimensional space and L2-normalized to produce the EMG embedding.

2) *Gaze Tracking*: A front-facing camera feeds a MediaPipe Face Mesh model [15] that extracts 468 facial landmarks at frame rate. From these, the system computes a 5-dimensional gaze feature vector from the displacement between the nose tip (landmark 4) and the eye center (midpoint of landmarks 33 and 362):

$$\mathbf{g} = [d_x, d_y, |d_x|, |d_y|, \sqrt{d_x^2 + d_y^2}] \quad (1)$$

where d_x and d_y are the horizontal and vertical displacements in normalized image coordinates. This vector captures both directional and magnitude information while remaining invariant to face scale. A dwell-based finite state machine maps sustained gaze fixation (exceeding 400ms on a target region) to one of five discrete AAC intent classes: *confirm*, *reject*, *scroll*, *select*, and *call-help*.

3) *Late Fusion Architecture*: The EMG embedding $\mathbf{e} \in \mathbb{R}^{64}$ and the gaze vector $\mathbf{g} \in \mathbb{R}^5$ are fused through a late fusion architecture. The gaze vector is first projected to a 64-dimensional space via a two-layer MLP, then concatenated with the EMG embedding to produce a 128-dimensional joint representation. A final two-layer classifier maps this to 5-class intent logits:

$$\hat{y} = \text{MLP}_{64 \rightarrow 5}(\text{ReLU}(\text{MLP}_{128 \rightarrow 64}([\mathbf{e}; \text{proj}(\mathbf{g})]))) \quad (2)$$

Late fusion was selected over a cross-attention alternative

based on a pre-specified decision criterion: cross-attention must exceed late fusion F1 by more than 3% (absolute) to justify its additional computational and parameter overhead. The ablation results (Section VI-B) did not meet this threshold.

D. Deterministic Context Scoring

We employ a deterministic, closed-form scoring function rather than a learned predictor to ensure bounded CPU and battery consumption during background polling. A semantic context c_i is scored against the current sensor snapshot as a convex combination of three environmental similarity scores:

$$S(c_i) = \alpha \cdot S_{\text{time}} + \beta \cdot S_{\text{gps}} + \gamma \cdot S_{\text{ble}} \quad (3)$$

where $\alpha + \beta + \gamma = 1$. The weights are derived from dynamic reliability estimates of each sensor signal (Section III-D4).

1) *Temporal Scoring*: The temporal similarity S_{time} applies a Gaussian kernel to the circular distance between the current time and the context’s expected activation time:

$$S_{\text{time}} = \exp\left(-\frac{d_{\text{circ}}(t, t_i)^2}{2\sigma^2}\right) \quad (4)$$

where d_{circ} handles the 24-hour wraparound and $\sigma = 2.0$ hours accommodates typical schedule variability.

2) *Spatial Scoring*: The spatial similarity S_{gps} applies an exponential decay to the Haversine great-circle distance between the current GPS coordinates and the context’s anchor location:

$$S_{\text{gps}} = \exp(-\lambda \cdot d_{\text{haversine}}) \quad (5)$$

with $\lambda = 0.1$ km (decay constant), establishing a contextual boundary of approximately 100 meters. At $d = 100$ m, the score decays to ≈ 0.37 ; at $d = 1$ km, it falls to near zero.

3) *BLE Proximity Scoring*: The BLE similarity S_{ble} evaluates the overlap between currently detected Bluetooth devices and a context’s registered device topology, with RSSI values mapped through a sigmoid function.

4) *Dynamic Reliability Normalization*: The weights α , β , and γ are computed dynamically from raw sensor reliability estimates rather than being statically configured. Time maintains a fixed baseline weight $w_t = 0.4$, guaranteeing that the normalization denominator $w_t + w_g + w_b$ is always positive even when both GPS and BLE are unavailable. GPS reliability w_g decays exponentially with reported accuracy ($w_g = \exp(-a/50.0)$ where a is the accuracy in meters), reflecting the degradation of location precision in indoor environments. BLE reliability w_b is computed from the mean sigmoid-transformed RSSI of detected devices, centered at -70 dBm. If GPS is unavailable ($w_g = 0$) or no BLE devices are detected ($w_b = 0$), the corresponding weight is zero and the remaining weights are renormalized to preserve the convex property $\alpha + \beta + \gamma = 1$. The value $w_t = 0.4$ was selected as an engineering heuristic to ensure meaningful temporal scoring under all sensor configurations; no systematic sensitivity analysis was performed. Contexts scoring below a dead-state threshold of 0.01 are excluded from routing consideration.

E. KV Cache Residency Management

The system maintains at most k KV cache states simultaneously resident in native memory, where k is configured based on the target device’s available RAM budget ($k = 3$ in the current deployment). KV cache states are mapped to native sequence slots managed by the underlying inference engine.

When a new context requires loading and all slots are occupied, the least-recently-used (LRU) state is evicted. Eviction is protected by a reference-counting mechanism: a state undergoing active inference holds a reference that blocks eviction until inference completes. The eviction loop waits for the reference count to drain to zero before releasing the native slot.

F. Asynchronous Drift Detection

Environmental sensors predict physical location but cannot detect whether the conversation topic has diverged from the cached context. A periodic background daemon (executing every 15 minutes via the Android WorkManager API) re-evaluates context scores using a lightweight sensor snapshot and compares them against currently resident states.

To prevent cache thrashing under noisy sensor conditions, state replacements are gated by a hysteresis margin: a candidate context must exceed the weakest resident state’s score by at least $\Delta = 0.10$ before a cache swap is authorized. This prevents oscillatory eviction-reload cycles that would impose unnecessary I/O and prefill costs.

A separate offline characterization study evaluates the sensitivity of *semantic* topic-shift detection—using SentenceBERT centroid distances on annotated DailyDialog conversations [18]—to the sliding window size k used for centroid smoothing (Section VI-E). This offline study is distinct from the deployed *DriftDetector*, which operates on sensor-score deltas rather than semantic embeddings. The offline ablation characterizes detection sensitivity as a function of k on available annotated data; the deployed system’s behavior is governed by the hysteresis margin Δ described above.

G. Inference Fallback

If a user triggers an intent before the background daemon has successfully pre-loaded any relevant context (a “cold miss”), the system employs a two-tier fallback pipeline. Tier 1 executes an immediate in-memory lookup of pre-configured generic responses and presents the result via text-to-speech output, maintaining immediate conversational responsiveness. Tier 2 initiates an asynchronous standard RAG prefill on the inference backend, allowing subsequent user intents to be upgraded to contextual inference once the prefill completes.

IV. IMPLEMENTATION

This section describes the engineering realization of the CAP-KVC system on an Android mobile device, including the JNI boundary design, native memory management, concurrency model, and sensor integration.

A. Platform and Runtime

AACBridge is implemented as a native Android application targeting arm64-v8a devices running Android 12 or later. The inference backend is built on `llama.cpp` [2], a C/C++ implementation of transformer inference optimized for consumer hardware. The model used in evaluation is Qwen2.5-0.5B-Instruct [3] quantized to Q4_K_M (4-bit) via `llama.cpp`'s GGUF format. Pre-compiled native libraries are bundled directly into the APK to avoid on-device compilation.

The Kotlin application layer communicates with the C++ inference backend through the Java Native Interface (JNI). All dependency injection is performed manually through a lightweight container class, avoiding framework overhead from Hilt, Dagger, or Koin to minimize startup latency.

B. JNI Boundary Contract

The JNI layer exposes nine native functions to the Kotlin runtime, implemented in a single C++ source file (`llama_jni.cpp`, 801 lines). Eight of these are application-facing; the ninth (`release`) handles lifecycle cleanup. A strict design constraint governs this boundary: **all arguments and return values are JNI scalar types** (`jstring`, `jint`, `jboolean`). No Java objects, arrays, or callbacks cross the JNI boundary. This eliminates the need for JNI global references, prevents garbage-collector root pinning, and simplifies the native lifecycle.

The nine exposed functions are organized into three functional groups:

- 1) **Lifecycle:** `initializeBackend()`, `initializeModel(path)`, and `release()`, which manage the native ggml context allocation and model weight loading.
- 2) **KV Cache Operations:** `saveKVCache(path, seqId)`, `loadKVCache(path, seqId)`, and `clearKVCache()`, which serialize, restore, and invalidate KV cache tensors associated with specific sequence slots.
- 3) **Inference:** `runInference(prompt)` for full prompt processing, `prefillOnly(prompt)` for context priming without generation, and `resumeInference(prompt)` for appending intent tokens to a previously loaded KV cache state.

The distinction between `runInference` and `resumeInference` is critical to the CAP-KVC design. `runInference` clears the existing session token history and processes tokens from position zero. `resumeInference` preserves the loaded token history and appends new tokens at position n_{past} , where n_{past} is the number of tokens already present in the restored KV cache. This prevents position-ID collisions that would corrupt the attention computation.

C. Concurrency Model

The C++ inference backend maintains two pieces of shared global state: a `llama_context` pointer and a `session_tokens` vector that tracks the token history for

KV cache consistency. Both are accessed by multiple Kotlin-side callers: the background context priming coroutine, the KV cache manager's load operations, and the foreground inference path triggered by user interaction.

To prevent data races, a single `ReentrantLock` (the "engine lock") is shared across all components that invoke JNI functions. This lock is held for the duration of any native call sequence that depends on consistent session state. Critically, the lock is held across *both* `loadKVCache` and `resumeInference` within a single acquisition, because releasing between calls would allow another thread to mutate `session_tokens`.

Within the KV cache manager, a separate per-state mutex system (using Kotlin coroutine `Mutex` instances) protects cache residency metadata. These per-state mutexes are orthogonal to the engine lock: they prevent concurrent load and eviction of the same semantic state without blocking unrelated cache operations.

D. KV Cache Priming Lifecycle

The context priming implementation executes a three-phase lifecycle per semantic state:

- 1) **Prefill:** The system acquires the engine lock and invokes `prefillOnly(prompt)`, which tokenizes the context prompt and runs the forward pass to populate the native KV cache with attention tensors. Unlike `runInference`, this function does not enter the token generation loop, producing clean cache state without stale generated tokens.
- 2) **Serialize:** While still holding the engine lock, the system invokes `saveKVCache(tmpPath, 0)` to serialize the KV tensors to a temporary file. Using sequence slot 0 is mandatory because `prefillOnly` always decodes into slot 0 via `llama_batch_get_one`.
- 3) **Atomic Commit:** After releasing the engine lock, the temporary file is atomically renamed to the final `.bin` path. On Android's ext4 filesystem, `rename()` is atomic, guaranteeing that the `.bin` file is either fully written or does not exist.

A model-readiness gate (`AtomicBoolean`) prevents priming attempts before the native context has been initialized, which would otherwise produce empty cache files containing only serialization headers.

E. KV Cache Residency and Eviction

The KV cache manager maintains a bounded pool of $k = 3$ native sequence slots, represented as a concurrent queue of available slot indices. When a cache load is requested and all slots are occupied, the manager identifies the least-recently-accessed state and initiates eviction.

Eviction follows a safe shutdown sequence: the victim state is first marked inactive (blocking new inference acquisitions), then the manager polls until its reference count reaches zero (using a 10ms delay to avoid CPU busy-spinning), and finally returns the native slot to the available pool. The current JNI layer does not expose an explicit `freeKVCache` function;

instead, reusing a sequence slot via `loadKVCache` implicitly overwrites the previous KV contents after the slot’s prior entries are cleared via `llama_memory_seq_rm`.

F. Sensor Integration

Background sensor acquisition is implemented through two cooperating components:

- **ActiveSweep:** A coroutine-based sensor collector that concurrently acquires GPS coordinates (via `LocationManager.getLastKnownLocation`) and BLE topology (via a 3000ms `BluetoothLeScanner` window). Both acquisitions run in parallel using Kotlin `async` and are combined into a `SensorSnapshot` for the state router.
- **DriftDetector:** A `CoroutineWorker` registered with Android’s `WorkManager` API for periodic execution every 15 minutes. To conserve battery during background polling, the drift detector uses a BLE-blind snapshot (empty device map), relying primarily on GPS and time signals. This is methodologically valid for hysteresis evaluation because both candidate and resident states are scored against the same BLE-blind snapshot, preserving relative score ordering.

A `BroadcastReceiver` registered for `BOOT_COMPLETED` and `MY_PACKAGE_REPLACED` intents ensures that the predictive cache is populated immediately after device boot or application update, using the `goAsync()` pattern to extend the receiver’s execution window.

Sensing cost. The sensor acquisition imposes minimal runtime overhead: GPS uses `getLastKnownLocation` (cached, no active fix), and BLE performs a 3-second scan bounded by `stopScan`. The `DriftDetector` polls every 15 minutes via `WorkManager`, which batches execution with other background work to minimize wake-lock durations. The primary energy cost of the system is the background KV cache priming (one full inference per cache state); this occurs at most once per context transition and executes only during idle periods when the device is charging or above a configurable battery threshold.

G. Fusion Model Deployment

The winning late fusion model is exported to ONNX format and bundled in the application’s `assets/` directory (`gaze_emg_fusion.onnx`, 45,459 bytes). On-device inference uses ONNX Runtime [30] with a single-threaded CPU execution provider. The session is lazily initialized at application scope to prevent repeated model loading during activity recreation events (e.g., screen rotation).

Input tensors are constructed from the EMG embedding (shape $1 \times 1 \times 64$, `float32`) and gaze feature vector (shape 1×5 , `float32`). The output logits are processed through an `argmax` operation followed by numerically stable softmax to extract both the predicted intent label and its associated confidence score. If the ONNX session fails to load or inference throws an exception, the system falls back to passing the raw gaze target through as the intent label with zero confidence.

H. Gaze Tracking

Gaze tracking uses the MediaPipe Face Landmarker task [15] loaded from a bundled `.task` model file (3.7 MB). The landmarker runs in live-stream mode on CameraX `ImageAnalysis` frames. The gaze displacement vector (Equation 1) is computed from nose-tip and eye-center landmarks and mapped to one of five screen quadrants plus a center region, each corresponding to an AAC intent class.

Intent firing follows a dwell-based finite state machine: a target must be fixated for at least 400ms before the intent is emitted, preventing accidental activations from saccadic eye movements. The dwell state machine executes on a single-threaded executor to guarantee sequential processing of asynchronous MediaPipe callbacks.

V. EXPERIMENTAL SETUP

A. Hardware Platform

All on-device benchmarks were conducted on a One-Plus 11R (Snapdragon 8+ Gen 1, 8 GB LPDDR5X, UFS 3.1 storage) running Android 14. The device was configured in airplane mode with display brightness fixed at 50% to minimize thermal and scheduling variability. No other foreground applications were active during benchmarking.

B. Language Model Configuration

The language model used throughout evaluation is Qwen2.5-0.5B-Instruct [3], quantized to Q4_K_M (4-bit mixed precision) via `llama.cpp`’s GGUF serialization format. This model was selected as the smallest instruction-tuned model in the Qwen2.5 family capable of producing coherent, context-aware responses on mobile hardware. The native context is configured with `n_ctx = 2048` (maximum sequence length), `n_batch = 512` (batch processing size), and `n_threads = 4` (inference thread count). Greedy decoding is used for all inference trials via `llama_sampler_init_greedy()`, which deterministically selects the highest-probability token at each step. The maximum generation length is 64 tokens, ensuring deterministic output sequences that minimize generation-phase variance across trials.

Table I summarizes all configurable system parameters for reproducibility.

C. Benchmark Conditions

The latency evaluation compares three pipeline configurations:

- 1) **ZERO_CONTEXT:** The language model receives only the user intent tokens (“User: I need water\nAssistant:”) with no environmental context. This establishes the hardware floor for inference latency on the target SoC.
- 2) **RAG_INLINE:** The full context prompt is concatenated with the user intent and submitted as a single string to `runInference()`, triggering a complete $O(N)$ token prefill at inference time. This represents the standard retrieval-augmented generation approach. Evaluated at

TABLE I
SYSTEM HYPERPARAMETERS. ALL VALUES ARE FIXED AT COMPILE TIME
EXCEPT THE MODEL PATH.

Component	Parameter	Value
LLM Engine	n_ctx (max sequence)	2048
	n_batch (batch size)	512
	n_threads	4
	Max generation tokens	64
Scoring	α (time baseline weight)	0.4
	β, γ (GPS, BLE)	dynamic
	Dead-state threshold	0.01
Drift	Hysteresis margin Δ	0.10
	Poll interval	15 min
Cache	Max active KV states	3
	BLE scan window	3000 ms
Benchmark	Warmup trials	2
	Measured trials	30

four prompt-length tiers: $N \in \{50, 100, 200, 500\}$ approximate tokens.

- 3) **CAP_KVC**: A pre-computed KV cache is restored from persistent storage via `loadKVCache()`, followed by `resumeInference()` with intent-only tokens. The latency reported includes both the cache restoration time and the subsequent generation time.

The CAP_KVC condition was evaluated using cache files primed from the base context prompts (~ 50 tokens). The N-scaling comparison therefore compares RAG_INLINE at increasing context lengths against CAP_KVC at a fixed cache size. This is by design: the CAP_KVC inference-time cost does not depend on the original context length under the evaluated configuration because the prefill was amortized during background priming. The cache restoration cost is dominated by file I/O (proportional to file size), not by attention computation.

Important limitation: Because the CAP_KVC cache states were derived from ~ 50 -token context prompts, comparisons against RAG_INLINE conditions with substantially longer contexts (e.g., $N \approx 500$) characterize *inference-time amortization behavior* rather than response quality under equivalent context richness. The speedup ratios quantify the latency advantage of cache restoration over repeated prefill, not the semantic equivalence of the generated responses.

D. Measurement Methodology

Each benchmark condition executes 32 sequential trials: 2 warmup trials (discarded) followed by 30 measured trials. The warmup phase absorbs JIT compilation of Kotlin/ART code paths and initial thermal settling. All measurements are wall-clock latency captured via `System.nanoTime()` at the Kotlin layer, wrapping the complete JNI call sequence.

Prefill isolation via native instrumentation. To decompose end-to-end latency into its constituent phases, `std::chrono::high_resolution_clock` timestamps are inserted immediately before and after each `llama_decode()` call within both `runInference` and `resumeInference`. This isolates the prefill phase—the single `llama_decode()` call that processes all prompt tokens through the transformer—from the subsequent autoregressive generation loop. The instrumentation also

records the generation-phase duration (from sampler initialization through the final generated token) and emits both timings, along with prompt and generated token counts, via `__android_log_print`.

On the Android NDK libcpp runtime, `std::chrono::high_resolution_clock` maps directly to the monotonic `std::chrono::steady_clock`, which is backed by the kernel’s `CLOCK_MONOTONIC` clock source. This prevents duration calculations from being corrupted by system NTP synchronization or manual clock adjustments. The overhead of the clock read call is ≈ 10 – 50 ns (leveraging the vDSO-accelerated kernel interface), which is negligible relative to the millisecond-scale model inference. JNI boundaries, string copies, and log serialization calls occur strictly outside the timed regions.

For the CAP_KVC condition, the benchmark instruments three sub-components separately: `cache_load_ms` (the `loadKVCache` call, measured at the Kotlin layer), `prefill_ms` (the intent-only `llama_decode` within `resumeInference`, measured at the native layer), and `gen_ms` (the generation loop, measured at the native layer). Both Kotlin-layer and native-layer measurements are captured under the same engine lock acquisition to prevent interleaving.

Results are emitted as structured CSV lines via Android’s `Log.i()` facility and extracted post-run via `adb logcat`. To mitigate the risk of packet drops or transient buffer overflows in Android’s logcat ring buffer, the post-run extraction script matches JNI timing entries to Kotlin bench entries by strict proximity and asserts alignment of trial indices and token counts. Any mismatch or incomplete sequence aborts the parser to ensure data integrity; the 180 trials reported here were fully verified and aligned.

E. Pre-Benchmark Validation

Before executing the benchmark suite, a semantic validation pass confirms that the three pipeline configurations produce qualitatively distinct outputs. A diagnostic prompt (“What should I have for breakfast and is my morning medication due?”) is submitted under each condition, and the generated text is logged for manual inspection. This validates that:

- ZERO_CONTEXT produces a generic response lacking environmental specificity.
- RAG_INLINE produces a response reflecting the `home_morning` context (breakfast, medication).
- CAP_KVC produces a similarly context-aware response, confirming that KV cache restoration successfully preserved the contextual attention patterns.

The benchmark suite is aborted if the KV cache load fails during validation, preventing the collection of meaningless data.

F. Memory Profiling

Native heap memory usage is measured using Android’s `dumpsys meminfo` command at two states: (1) after model initialization with zero KV cache states loaded, and (2) after

loading three KV cache states into native sequence slots. The delta represents the marginal memory cost of the predictive caching system.

G. Fusion Model Evaluation

The multimodal fusion model is evaluated through a controlled ablation study comparing two architectures:

- **Cross-Attention:** A single-head cross-attention module ($d = 64$) where the EMG embedding serves as the query and the projected gaze vector serves as key and value.
- **Late Fusion:** Concatenation of the 64-dimensional EMG embedding and the 64-dimensional projected gaze vector, followed by a two-layer MLP classifier.

Both architectures are trained on a synthetic dataset of 2000 paired EMG-gaze samples (60% train, 20% validation, 20% test) drawn from a synthetic data generator that pairs random EMG embeddings with corresponding gaze vectors. The pre-specified decision criterion requires that the cross-attention model exceed the late fusion model’s macro F1 score by more than 3 percentage points (absolute) to justify deployment of the more complex architecture.

Limitation: The fusion training data is synthetically generated and does not represent real paired EMG-gaze recordings from AAC users. The reported accuracy and F1 scores characterize the model’s capacity to learn the fusion mapping under controlled conditions, not its expected clinical accuracy. This is explicitly acknowledged as a limitation of the current evaluation.

H. EMG Classifier Evaluation

The CNN-LSTM EMG classifier is trained on a subset of the NinaPro DB5 dataset [14] using a leave-one-subject-out (LOSO) protocol: subjects 1–9 for training and subject 10 for validation. Training uses the Adam optimizer with learning rate 10^{-3} , cosine annealing schedule, class-weighted cross-entropy loss, and gradient clipping at norm 1.0. The model is trained for up to 50 epochs with early stopping based on validation accuracy.

The EMG classifier operates as an embedding extractor for the downstream fusion model. Its standalone classification accuracy on the held-out subject is reported separately from the fusion accuracy to avoid conflation.

I. Drift Detection Ablation

The sensitivity of conversation-level drift detection to the sliding window size k is evaluated offline on the DailyDialog dataset [18]. Each conversation is annotated with ground-truth topic shift indices. For each $k \in \{1, 2, 3, 4, 5\}$, the system computes the centroid embedding of the most recent k turns using Sentence-BERT (all-MiniLM-L6-v2 [19]), and predicts a topic shift when the cosine similarity between the anchor embedding and the centroid falls below a threshold $\theta = 0.15$. Precision, recall, and F1 are computed for each k . The MiniLM model is used solely for this offline ablation analysis and is not part of the deployed Android runtime; the

on-device drift detector operates on router score deltas rather than semantic embeddings.

Limitation: DailyDialog is a general-purpose dialogue corpus that differs substantially from AAC communication patterns in terms of turn length, vocabulary, and topic variability. The k-ablation results characterize the detector’s behavior on available annotated data rather than providing direct evidence of clinical effectiveness.

VI. RESULTS

A. End-to-End Inference Latency (RQ1)

Table II summarizes the end-to-end inference latency across all benchmark conditions ($n = 30$ measured trials per condition, 180 total measurements).

TABLE II
INFERENCE LATENCY DECOMPOSITION (MS) ACROSS PIPELINE CONDITIONS. PREFILL ISOLATES THE `LLAMA_DECODE()` FORWARD PASS; GENERATION COVERS THE AUTOREGRESSIVE SAMPLING LOOP. BOTH ARE MEASURED VIA NATIVE-SIDE `STD::CHRONO` INSTRUMENTATION. CAP_KVC TOTAL INCLUDES CACHE RESTORATION, PREFILL, AND GENERATION. ALL VALUES: ONEPLUS 11R, QWEN2.5-0.5B-INSTRUCT Q4_K_M, GREEDY SAMPLING, MAX 64 OUTPUT TOKENS, $n = 30$ TRIALS.

Condition	E2E (ms)	Prefill (ms)	Gen (ms)	E2E σ	Tokens
ZERO_CONTEXT	3823.78	274.79	3544.58	569.76	9
RAG ($N \approx 50$)	6547.13	2021.46	4508.53	8392.58	52
RAG ($N \approx 100$)	7900.92	3015.09	4877.95	3857.18	89
RAG ($N \approx 200$)	12122.30	6673.53	5439.96	2740.12	206
RAG ($N \approx 500$)	19408.89	14001.46	5390.76	2113.04	439
CAP_KVC (cache load)	2.74	—	—	1.35	—
CAP_KVC (inference)	4571.97	343.35	4226.74	373.65	9
CAP_KVC (total)	4574.70	—	—	373.49	—

Fig. 2 visualizes these results, and Fig. 3 provides a detailed breakdown showing the stacked composition of CAP-KVC latency.

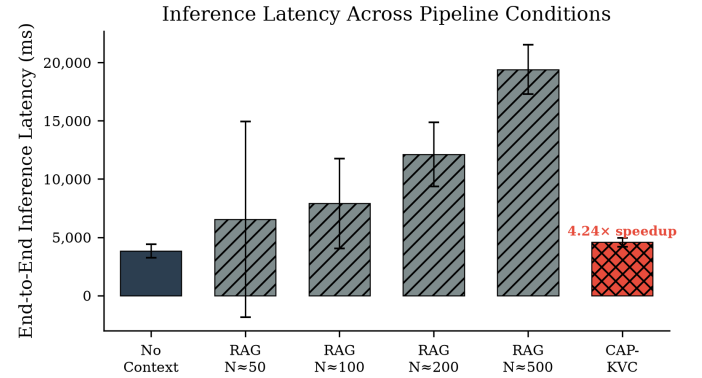


Fig. 2. End-to-end inference latency across pipeline conditions ($n = 30$ trials each). Error bars indicate one standard deviation. CAP-KVC achieves a $4.24\times$ end-to-end speedup relative to RAG_INLINE at $N \approx 500$.

Several observations follow from these results:

Cache restoration is negligible. The mean cache load time is 2.74 ms ($\sigma = 1.35$ ms), representing less than 0.1% of the total CAP_KVC latency. This duration measures the complete Kotlin-side `loadKVCache()` round trip, including JNI marshalling, native file I/O, and KV tensor deserialization. The restoration is dominated by hardware I/O and memory copy rather than computational processing.

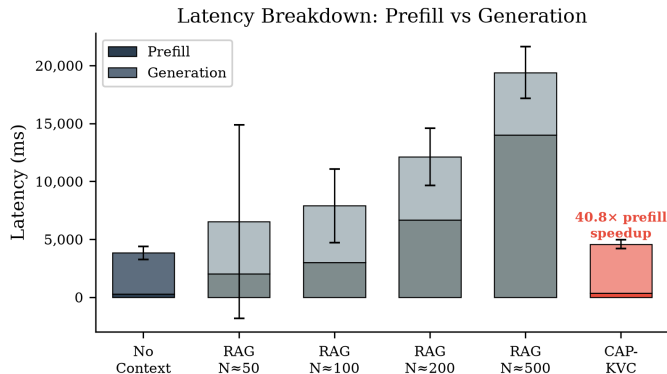


Fig. 3. Latency breakdown across pipeline conditions. Native-side `std::chrono` instrumentation decomposes each trial into prefill (dark) and generation (light) phases. RAG_INLINE prefill scales linearly with context length (274 ms at $N=9$ to 14001 ms at $N=439$ tokens), whereas CAP_KVC prefill is 343 ms regardless of cached context size, achieving a $40.78\times$ prefill-specific speedup. Error bars: one standard deviation ($n=30$).

CAP_KVC avoids context-proportional prefill. The native-side prefill decomposition (Table II) confirms that RAG_INLINE prefill scales linearly with prompt length: from 2021 ms at $N \approx 52$ tokens to 14001 ms at $N \approx 439$ tokens, a $6.93\times$ increase. In contrast, the CAP_KVC prefill cost is 343.35 ms ($\sigma = 45.00$ ms) for 9 intent-only tokens, regardless of the cached context size. This represents a **$40.78\times$ prefill speedup** relative to RAG_INLINE at $N \approx 500$.

Speedup relative to RAG_INLINE. End-to-end, CAP_KVC achieves a total latency of 4574.70 ms versus RAG_INLINE’s 19408.89 ms at $N \approx 500$, an end-to-end speedup of $4.24\times$. At $N \approx 50$, the mean end-to-end speedup is $1.43\times$ ($6547.13 / 4574.70$), though this end-to-end difference is not statistically significant ($p = 0.209$) due to high variance in the RAG_INLINE generation phase; the prefill-specific speedup at $N \approx 50$ remains statistically significant ($p = 0.011$). The prefill decomposition reveals that the end-to-end speedup is heavily masked by the generation phase, which is comparable across conditions (~ 3500 – 5400 ms).

Statistical significance and assumptions. Welch’s two-sample t -tests ($\alpha = 0.05$, two-tailed) on the *prefill-only* latency confirm statistical significance: CAP_KVC versus RAG_INLINE at $N \approx 50$ ($t(29.0) = -2.71$, $p = 0.011$, Cohen’s $d = 0.70$) and CAP_KVC versus RAG_INLINE at $N \approx 500$ ($t(29.0) = -40.42$, $p < 10^{-26}$, Cohen’s $d = 10.44$). The end-to-end comparison at $N \approx 500$ is also significant ($t(30.8) = -37.86$, $p < 10^{-26}$, $d = 9.78$).

Although Welch’s t -test is robust to unequal variances (which are expected due to prompt-dependent prefill scaling), the extreme skewness of the underlying RAG latency distributions violates standard parametric normality assumptions. The resulting t -test statistics should be interpreted as general trends rather than exact probabilities. For $N \approx 50$ prefill, Cohen’s $d = 0.70$ indicates a medium-to-large effect. For $N \approx 500$, $d = 10.44$ represents an overwhelming effect. Cohen’s d was computed using equal-sample pooled variance; we note that this value is attenuated by the high variance of the RAG

condition.

Outlier analysis. For diagnostic purposes, Tukey’s outlier criterion ($Q_3 + 3 \times \text{IQR}$) was applied to identify extreme latency anomalies. This flagged 5 outlier trials (3 in RAG@50, 1 in ZERO_CONTEXT, 1 in CAP_KVC) exhibiting latency substantially above the condition mean. The precise mechanism underlying these outliers—whether OS-level thermal management, background scheduler preemption, garbage collection pauses, or other system-level effects—was not independently measured during the benchmark run. No thermal telemetry, CPU frequency logs, or scheduler event traces were captured alongside the benchmark data. To ensure scientific conservatism and preserve the operational reality of edge hardware, these outliers were not removed from the statistics reported in Table II or the primary t -test calculations.

Generation-phase analysis. The generation-phase duration varies across conditions: ZERO_CONTEXT 3544 ms (58 tokens), RAG_INLINE@50 4508 ms (64 tokens), RAG_INLINE@500 5390 ms (64 tokens), and CAP_KVC 4226 ms (64 tokens). The variation in generation time is consistent with two factors: (1) differences in per-token attention computation cost when the KV cache contains different numbers of entries, and (2) system-level effects associated with sustained mobile execution over a 29-minute benchmark session (the benchmark captured no thermal telemetry or CPU frequency data to distinguish specific mechanisms). Critically, because prefill and generation are now decomposed, these generation-phase differences do not confound the prefill speedup measurement.

B. Multimodal Fusion Ablation (RQ3)

Table III presents the ablation comparison between the two candidate fusion architectures, evaluated on the synthetic test set (400 samples).

TABLE III
FUSION ARCHITECTURE ABLATION RESULTS. VALUES FROM FUSION_ABLATION.CSV. LATENCY IS PER-SAMPLE CPU INFERENCE TIME.

Architecture	Accuracy	F1 (Macro)	Precision	Recall
Cross-Attention	0.9467	0.9437	0.9459	0.9450
Late Fusion	0.9967	0.9963	0.9956	0.9972

Late fusion achieves a macro F1 of 0.9963 compared to 0.9437 for cross-attention, a difference of 5.26 percentage points. Because the cross-attention model *underperforms* the simpler architecture rather than exceeding it, the pre-specified 3% superiority threshold is not met, and the late fusion model is selected for deployment. The automated decision logic writes the winning architecture identifier to a file (`winner.txt`), and the corresponding model checkpoint (`late_fusion_best.pth`, 48 KB) is exported to ONNX format and bundled in the application assets. Fig. 4 visualizes the comparison.

Synthetic data caveat. The training and evaluation data are synthetically generated, pairing random EMG embeddings with gaze vectors. The high accuracy reflects the model’s capacity to learn the intended fusion mapping under controlled

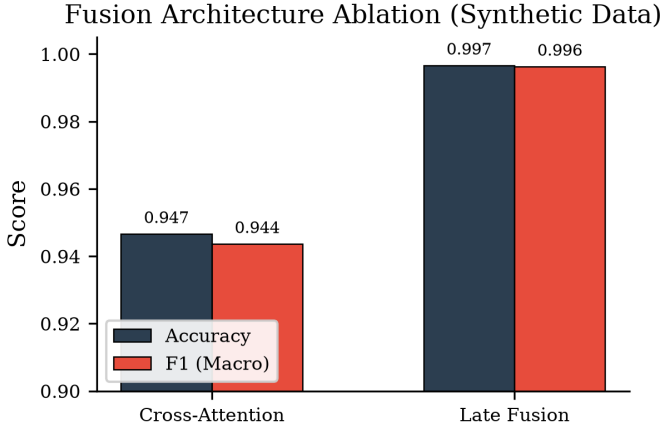


Fig. 4. Fusion architecture ablation on synthetic test data (400 samples). Late fusion exceeds cross-attention in both accuracy and F1.

conditions and validates that the late fusion architecture can successfully combine heterogeneous embedding spaces of differing dimensionality ($\mathbb{R}^{64}$ and $\mathbb{R}^5$). These numbers should not be interpreted as predictions of clinical intent-recognition accuracy on real paired EMG-gaze recordings from AAC users, where sensor noise, electrode placement variability, and user-specific physiological differences would substantially affect performance.

C. EMG Classifier Performance

The standalone CNN-LSTM EMG classifier, evaluated on the held-out subject (LOSO protocol, subject 10), achieves a test accuracy of 42.4% with a macro F1 of 0.37. Table IV shows the per-class breakdown.

TABLE IV
EMG STANDALONE CLASSIFICATION REPORT (HELD-OUT SUBJECT).
VALUES FROM `CLASSIFICATION_REPORT.TXT`.

Class	Precision	Recall	F1	Support
confirm	0.25	0.11	0.15	18
reject	0.00	0.00	0.00	18
scroll	0.42	0.72	0.53	18
select	0.77	0.50	0.61	20
call-help	0.45	0.78	0.57	18
Macro avg	0.38	0.42	0.37	92

The low standalone accuracy is expected under cross-subject evaluation, where inter-subject variability in facial EMG patterns is a well-documented challenge [14]. The *reject* class is entirely unrecognized (0% recall), and the model exhibits a strong bias toward the *scroll* and *call-help* classes, as visible in the confusion matrix (Fig. 6). The training curves (Fig. 5) show clear overfitting: training accuracy reaches approximately 80% by epoch 35 while validation accuracy plateaus near 40%, with the best checkpoint selected at epoch 8 (validation accuracy $\approx 47\%$).

These results demonstrate that the standalone EMG classifier does not achieve sufficient accuracy for deployment as a primary intent recognizer under cross-subject conditions. The multimodal fusion approach compensates by combining the EMG embedding with gaze tracking, achieving higher

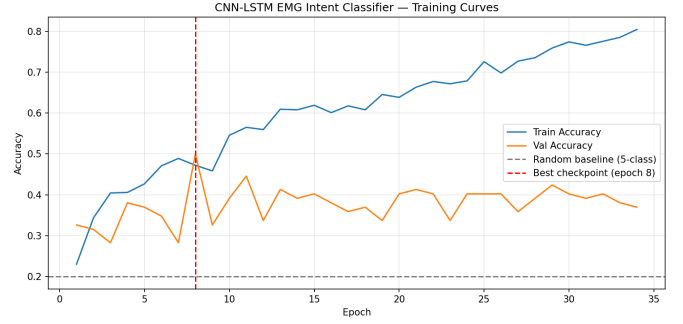


Fig. 5. CNN-LSTM EMG classifier training and validation accuracy over 35 epochs. The red dashed line marks the best checkpoint at epoch 8. The gray dashed line indicates the 5-class random baseline (20%).

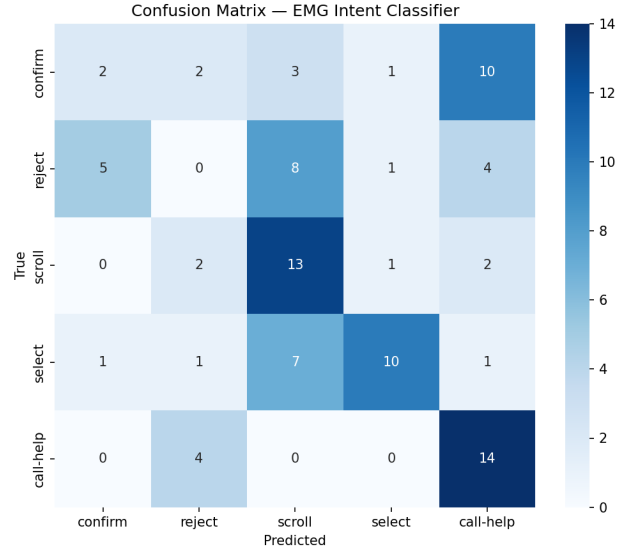


Fig. 6. Confusion matrix for the EMG standalone classifier on the held-out subject. Strong *call-help* and *scroll* diagonal entries contrast with near-zero *confirm* and *reject* performance.

classification performance on the synthetic evaluation set (Section VI-B). However, no gaze-only baseline was evaluated, so the claim that fusion improves upon both unimodal channels independently is **not supported** by the current experimental design. The on-device CNN-LSTM inference latency is 1.98 ms (mean, $\sigma = 1.12$ ms, $n = 200$ trials), indicating that embedding extraction is feasible within real-time constraints.

D. Memory Budget (RQ2)

Table V reports native heap usage measured via `dumpsys meminfo` at two operating points.

TABLE V
NATIVE HEAP MEMORY USAGE (KB) FROM `DUMPSYS MEMINFO`. DELTA REPRESENTS THE MARGINAL COST OF LOADING THREE KV CACHE STATES.

State	Native Heap (KB)	Total PSS (KB)
Idle (0 states)	18,856	623,488
Loaded (3 states)	140,420	612,940
Delta	+121,564 (≈ 118.7 MB)	—

The three-state KV cache residency adds approximately 118.7 MB of native heap (121,564 KiB $\div$ 1024). The total PSS (proportional set size) decreases slightly between states due to measurement variability in shared memory attribution. The native heap growth is consistent with three copies of the model’s KV cache tensors ($\sim$ 40 MB per state for a 0.5B-parameter model at Q4_K_M quantization). Fig. 7 visualizes the two operating points.

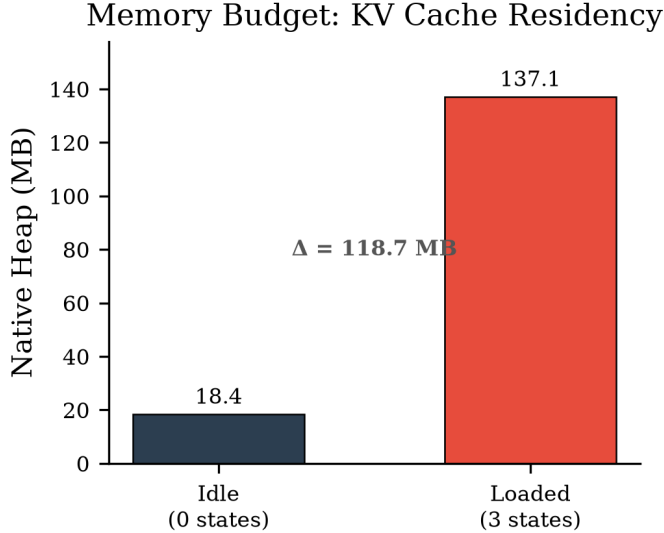


Fig. 7. Native heap memory usage at idle (0 KV states) and with 3 KV cache states loaded. The delta of 118.7 MB represents the marginal cost of predictive caching.

E. Drift Detection Ablation

Table VI presents the k -ablation results for the offline semantic drift detection characterization on annotated DailyDialog conversations ($\theta = 0.15$). This evaluation uses Sentence-BERT centroid distances [19] to detect topic shifts in text transcripts and is methodologically distinct from the deployed DriftDetector, which operates on environmental sensor-score deltas (Section III-F).

TABLE VI
DRIFT DETECTION F1 AS A FUNCTION OF SLIDING WINDOW SIZE k , EVALUATED ON DAILYDIALOG WITH $\theta = 0.15$. n_{TRUE} AND n_{PRED} DENOTE THE NUMBER OF GROUND-TRUTH AND PREDICTED SHIFT POINTS, RESPECTIVELY.

k	θ	Precision	Recall	F1	n_{true}	n_{pred}
1	0.15	0.367	0.367	0.367	150	150
2	0.15	0.341	0.313	0.326	150	138
3	0.15	0.137	0.107	0.120	150	117
4	0.15	0.071	0.040	0.051	150	84
5	0.15	0.039	0.013	0.020	150	52

The highest F1 (0.367) is achieved at $k = 1$, with performance degrading monotonically as the window size increases (Fig. 8). This indicates that the centroid smoothing introduced by larger k values reduces sensitivity to genuine topic shifts under the selected threshold. However, the absolute F1 values are low across all k (ranging from 0.367 to 0.020), reflecting both the inherent difficulty of unsupervised topic-shift

detection and the substantial mismatch between DailyDialog’s multi-turn conversational dynamics and the sparse, short-turn interactions typical of AAC usage.

Deployment rationale for $k = 3$. The deployed DriftDetector uses $k = 3$ for its sensor-score smoothing window. The DailyDialog ablation operates on a different detection mechanism (Sentence-BERT centroid distances) and cannot directly inform the deployed sensor-score threshold. The value $k = 3$ was retained as an engineering smoothing heuristic: in the mobile environment, sensor score sequences are noisier than annotated dialogue transcripts, and a larger window provides stability against transient fluctuations. The choice should be understood as a conservative design decision rather than an empirically validated parameter.

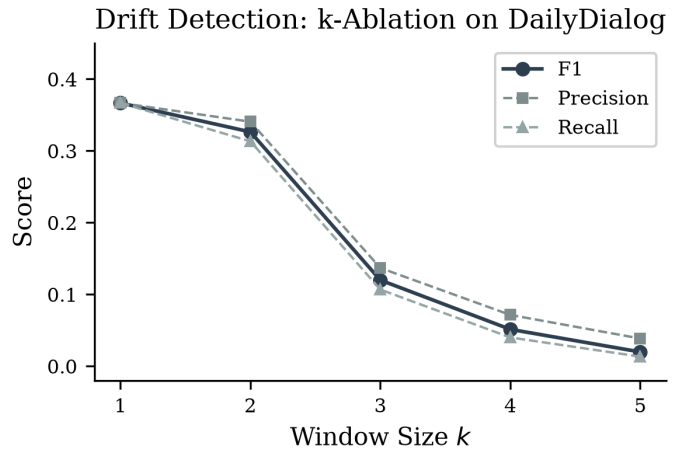


Fig. 8. Drift detection precision, recall, and F1 as a function of sliding window size k on DailyDialog ($\theta = 0.15$). Performance degrades monotonically with increasing k .

F. Summary of Key Findings

The experimental results support the following conclusions:

- 1) CAP-KVC achieves a $40.78\times$ prefill-specific speedup at $N \approx 500$ tokens (343 ms vs. 14001 ms), confirmed by native-side `std::chrono` instrumentation. The end-to-end speedup is $4.24\times$ (4574 ms vs. 19408 ms), with the generation phase ($\sim$ 4000–5400 ms) accounting for the difference between prefill and end-to-end ratios. Cache restoration is 2.74 ms (negligible).
- 2) The late fusion architecture achieves a macro F1 of 0.9963 on synthetic evaluation data, exceeding cross-attention (F1 = 0.9437) while requiring fewer parameters and lower inference latency, supporting the pre-specified selection criterion.
- 3) The standalone EMG classifier achieves 42.4% accuracy under cross-subject evaluation, motivating multimodal fusion as a necessary complement rather than a standalone solution.
- 4) The three-state KV cache residency requires approximately 118.7 MB of additional native heap, a feasible budget on modern mobile devices with 8 GB RAM.
- 5) Drift detection on DailyDialog yields a maximum F1 of 0.367 ($k = 1$), suggesting that the chosen threshold

($\theta = 0.15$) and evaluation corpus may require further calibration for clinical AAC deployment.

VII. CONCLUSION

This paper presented CAP-KVC, a system that amortizes context prefill costs for language model inference on mobile hardware by predicting environmental context from sensor signals and pre-computing KV cache states during idle periods. On a OnePlus 11R with Qwen2.5-0.5B-Instruct (Q4_K_M), native-side prefill decomposition confirms a $40.78\times$ prefill-specific speedup at $N \approx 500$ tokens: CAP-KVC processes 9 intent tokens (343 ms) versus 439 tokens for inline processing (14001 ms). End-to-end latency is reduced by $4.24\times$ (4574 ms vs. 19408 ms). Cache restoration from persistent storage completes in 2.74 ms on average, confirming that the amortized prefill cost is negligible at inference time.

The system is implemented as a deployable Android application with a scalar-only JNI boundary (9 native functions), a reentrant-lock concurrency model, and a bounded KV cache residency manager operating within a 118.7 MB native heap budget for three simultaneous cache states. A proof-of-concept multimodal intent layer combining sEMG and gaze inputs through late fusion demonstrates architectural feasibility on synthetic evaluation data (macro F1 = 0.9963); this component requires clinical validation before deployment.

The primary contribution is the demonstration that sensor-driven predictive caching can eliminate prompt-length-dependent prefill latency on resource-constrained edge devices. The approach is applicable beyond the AAC scenario used as a motivating example: any application where a mobile language model processes recurring environmental contexts can benefit from idle-time cache priming.

A. Limitations

Several limitations constrain the generalizability of these results and should be addressed in future evaluations:

- 1) **Single-device, limited-sample evaluation.** All benchmarks were conducted on a single device (OnePlus 11R, Snapdragon 8+ Gen 1) with $n = 30$ measured trials per condition. Latency characteristics, thermal behavior, and memory budgets may differ on other mobile SoCs, and the moderate sample size limits the precision of variance estimates.
- 2) **Single model.** Only Qwen2.5-0.5B-Instruct (Q4_K_M) was evaluated. The relationship between CAP-KVC speedup and model size, quantization level, or architecture family has not been characterized.
- 3) **Synthetic fusion data.** The fusion model was trained and evaluated on synthetically generated EMG-gaze pairs. The reported macro F1 of 0.9963 reflects model capacity under controlled conditions and does not predict clinical accuracy. No gaze-only unimodal baseline was evaluated.
- 4) **Latency metric scope.** The prefill timing (Table II) measures the single `llama_decode()` batch-decode call via native-side `std::chrono` instrumentation. Tokenization and batch construction overhead (estimated

< 1 ms for prompts up to 500 tokens) are excluded. Generation-phase duration varies across conditions due to differing KV cache sizes affecting per-token attention cost and uncharacterized system-level effects during the 29-minute benchmark session. No thermal telemetry, CPU frequency logs, or OS scheduler traces were captured during the benchmark run, so the specific mechanisms underlying generation-phase variance cannot be attributed. Because prefill and generation are decomposed independently, these generation-phase differences do not confound the prefill speedup measurement.

- 5) **Static context configurations.** The benchmarking uses five pre-configured semantic contexts rather than dynamically learned user profiles. A production deployment would require user-configurable contexts and longitudinal evaluation of context relevance.
- 6) **No non-predictive cache baseline.** The evaluation compares CAP-KVC against inline context processing (RAG_INLINE) and zero-context inference but does not include a static cache baseline (i.e., a fixed pre-loaded KV cache without prediction or drift detection). Such a baseline would isolate the contribution of the predictive routing and drift components from the raw benefit of cache restoration itself.
- 7) **Drift detection.** The offline Sentence-BERT k-ablation yields a maximum F1 of 0.367 ($k = 1$) on DailyDialog, while the deployed sensor-score-based system uses $k = 3$. These are methodologically different detection mechanisms, and the DailyDialog results cannot directly validate the deployed sensor-score hysteresis. Both the window size ($k = 3$) and threshold ($\theta = 0.15$, $\Delta = 0.10$) are engineering heuristics.
- 8) **No clinical evaluation.** The system has not been evaluated with AAC users in clinical or naturalistic settings. User satisfaction, communicative effectiveness, and caregiver acceptance remain to be assessed.
- 9) **No thermal or power telemetry.** The benchmark run captured only application-level timing data via `adb logcat` filtered to the AACBridge process. No thermal sensor readings, CPU frequency logs, OS-level thermal management events, or power consumption measurements were collected during the 29-minute benchmark session. Consequently, the specific mechanisms underlying latency outliers and generation-phase variance cannot be attributed to thermal throttling versus other system-level effects.
- 10) **No cache file encryption.** Serialized KV cache tensors are stored as raw binary files in the application’s internal storage directory. Protection relies entirely on the Android application sandbox; no additional encryption or integrity verification is applied. In threat models involving rooted devices or physical access, cached KV tensors—which encode the semantic content of contextual prompts—could be extracted.
- 11) **Battery impact not characterized.** The `ContextDaemon` foreground service (60-second sweep interval) and the `DriftDetector` periodic worker (15-minute interval via `WorkManager`) impose

recurring sensor acquisition and, when cache states require priming, full model inference. The aggregate battery impact of these background operations has not been measured. No WakeLock, PowerManager, or battery-level gating is implemented beyond the Android system’s standard WorkManager batching behavior and the daemon’s `onLowMemory()` sweep cancellation. Additionally, the coroutine-based `delay()` scheduling in `ContextDaemon` does not hold a WakeLock, so sweep intervals may be deferred by Android Doze mode when the device is idle and stationary.

Despite these limitations, the results demonstrate that predictive KV cache priming is a viable strategy for reducing context-dependent inference latency on resource-constrained mobile hardware, and that the cache restoration cost (~ 3 ms) is negligible relative to the generation phase.

B. Artifact Availability

The source code, benchmark scripts, and raw benchmark logs are publicly available at <https://github.com/omni-ar/AACBridge>.

REFERENCES

- [1] D. R. Beukelman and J. C. Light, “Augmentative and alternative communication: Supporting children and adults with complex communication needs,” *Paul H. Brookes Publishing*, 5th ed., 2020.
- [2] G. Gerganov, “llama.cpp: LLM inference in C/C++,” 2023. [Online]. Available: <https://github.com/ggerganov/llama.cpp>
- [3] A. Yang, B. Yang, B. Hui, B. Zheng, B. Yu, et al., “Qwen2.5 Technical Report,” *arXiv preprint arXiv:2412.15115*, 2024.
- [4] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer, “LLM.int8(): 8-bit matrix multiplication for transformers at scale,” in *Proc. NeurIPS*, 2022.
- [5] E. Frantar, S. Ashkboos, T. Hoeffler, and D. Alistarh, “GPTQ: Accurate post-training quantization for generative pre-trained transformers,” in *Proc. ICLR*, 2023.
- [6] MLC Team, “MLC LLM: Universal LLM deployment engine,” 2023. [Online]. Available: <https://github.com/mlc-ai/mlc-llm>
- [7] Z. Liu, A. Desai, F. Liao, W. Wang, V. Xie, Z. Xu, A. Kyrillidis, and A. Shrivastava, “Scissorhands: Exploiting the persistence of importance hypothesis for LLM KV cache compression at test time,” in *Proc. NeurIPS*, 2023.
- [8] Z. Zhang, Y. Sheng, T. Zhou, T. Chen, L. Zheng, R. Cai, Z. Song, Y. Tian, C. Ré, C. Barrett, Z. Wang, and B. Chen, “H₂O: Heavy-hitter oracle for efficient generative inference of large language models,” in *Proc. NeurIPS*, 2023.
- [9] G. Xiao, Y. Tian, B. Chen, S. Han, and M. Lewis, “Efficient streaming language models with attention sinks,” *arXiv preprint arXiv:2309.17453*, 2023.
- [10] L. Zheng, L. Yin, Z. Xie, C. Sun, J. Huang, C. H. Yu, S. Cao, C. Kozyrakis, I. Stoica, J. E. Gonzalez, C. W. Barrett, and Y. Sheng, “SGLang: Efficient execution of structured language model programs,” in *Proc. NeurIPS*, 2024.
- [11] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, “Efficient memory management for large language model serving with PagedAttention,” in *Proc. 29th SOSP*, 2023.
- [12] W. Lee, J. Lee, J. Seo, and J. Sim, “InfiniGen: Efficient generative inference of large language models with dynamic KV cache management,” in *Proc. 18th USENIX OSDI*, 2024.
- [13] Y. Liu, H. Li, Y. Cheng, S. Ray, Y. Huang, Q. Zhang, K. Du, J. Yao, S. Lu, G. Ananthanarayanan, M. Maire, H. Hoffmann, A. Holtzman, and J. Jiang, “CacheGen: KV cache compression and streaming for fast large language model serving,” in *Proc. ACM SIGCOMM*, 2024.
- [14] M. Atzori, A. Gijsberts, C. Castellini, B. Caputo, A.-G. M. Hager, S. Elsig, G. Giatsidis, F. Bassetto, and H. Müller, “Electromyography data for non-invasive naturally-controlled robotic hand prostheses,” *Scientific Data*, vol. 1, p. 140053, 2014.
- [15] C. Lugaresi, J. Tang, H. Nash, C. McClanahan, E. Uboweja, M. Hays, F. Zhang, C.-L. Chang, M. G. Yong, J. Lee, et al., “MediaPipe: A framework for building perception pipelines,” 2019. [Online]. Available: <https://mediapipe.dev>
- [16] K. Vertanen and P. O. Kristensson, “Complementing text entry evaluations with a composition task,” *ACM Trans. Comput.-Hum. Interact.*, vol. 21, no. 5, pp. 1–33, 2015.
- [17] J. Ngiam, A. Khosla, M. Kim, J. Nam, H. Lee, and A. Y. Ng, “Multimodal deep learning,” in *Proc. 28th ICML*, 2011.
- [18] Y. Li, H. Su, X. Shen, W. Li, Z. Cao, and S. Niu, “DailyDialog: A manually labelled multi-turn dialogue dataset,” *arXiv preprint arXiv:1710.03957*, 2017.
- [19] W. Wang, F. Wei, L. Dong, H. Bao, N. Yang, and M. Zhou, “MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers,” in *Proc. NeurIPS*, 2020.
- [20] Y. Li, Y. Huang, B. Yang, B. Venkitesh, A. Locatelli, H. Ye, T. Cai, P. Lewis, and D. Chen, “SnapKV: LLM Knows What You are Looking for Before Generation,” *arXiv preprint arXiv:2404.14469*, 2024.
- [21] I. Gim, G. Chen, S.-J. Lee, N. Sarda, A. Khandelwal, and L. Zhong, “Prompt Cache: Modular Attention Reuse for Low-Latency Inference,” in *Proc. MLSys*, 2024.
- [22] J. Yao, H. Li, Y. Liu, S. Ray, Y. Huang, Q. Zhang, K. Du, S. Lu, G. Ananthanarayanan, M. Maire, H. Hoffmann, A. Holtzman, and J. Jiang, “CacheBlend: Fast Large Language Model Serving for RAG with Cached Knowledge Fusion,” *arXiv preprint arXiv:2405.16444*, 2024.
- [23] Z. Xue, Y. Huang, S. Du, L. Lu, H. Chen, J. Wen, Y. Zhang, X. Li, Z. Li, W. Zhang, X. Zhuang, L. Xia, Z. Xu, S. Song, Y. Xu, D. Shu, L. Yuan, and H. Chen, “PowerInfer-2: Fast Large Language Model Inference on a Smartphone,” *arXiv preprint arXiv:2406.06282*, 2024.
- [24] C. Hooper, S. Kim, H. Mohammadzadeh, M. W. Mahoney, Y. S. Shao, K. Keutzer, and A. Gholami, “KVQuant: Towards 10 Million Context Length LLM Inference with KV Cache Quantization,” in *Proc. NeurIPS*, 2024.
- [25] J. Lin, J. Tang, H. Tang, S. Yang, W.-M. Chen, W.-C. Wang, G. Xiao, X. Dang, C. Gan, and S. Han, “AWQ: Activation-Aware Weight Quantization for LLM Compression and Acceleration,” in *Proc. MLSys*, 2024.
- [26] Z. Liu, C. Zhao, F. Iandola, C. Lai, Y. Tian, I. Fedorov, Y. Xiong, E. Chang, Y. Shi, R. Krishnamoorthi, L. Lai, and V. Chandra, “MobileLLM: Optimizing Sub-billion Parameter Language Models for On-Device Use Cases,” in *Proc. ICML*, 2024.
- [27] K. Alizadeh, I. Mirzadeh, D. Belenko, K. Khatamifard, M. Cho, C. C. Del Mundo, M. Rastegari, and M. Farajtabar, “LLM in a Flash: Efficient Large Language Model Inference with Limited Memory,” *arXiv preprint arXiv:2312.11514*, 2023.
- [28] Y. Sheng, L. Zheng, B. Yuan, Z. Li, M. Ryabinin, B. Chen, P. Liang, C. Ré, I. Stoica, and C. Zhang, “FlexGen: High-Throughput Generative Inference of Large Language Models with a Single GPU,” in *Proc. ICML*, 2023.
- [29] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, “Attention is all you need,” in *Proc. NeurIPS*, 2017.
- [30] ONNX Runtime Contributors, “ONNX Runtime,” 2021. [Online]. Available: <https://onnxruntime.ai>